Simulation of Large-scale Fading

Date Performed:
Expt. No.:
Reg. No.:
Name:

1 Objective

To study large-scale fading with the aid of simulations based on MATLAB or IT++ library.

2 Equipments/Tool(s) Required

MATLAB OR IT++

3 Theory

3.1 Large-scale Fading

3.1.1 General Path Loss Model

The free-space propagation model is used for predicting the received signal strength in the line of-sight (LOS) environment where there is no obstacle between the transmitter and receiver. When non-isotropic antennas are used with a transmit gain of G_t and a receive gain of G_r, the received power at distance d, $P_r(d)$, is expressed by the well-known Friis equation [1], given as

$$P_r(d) = \frac{P_t G_t G_r \lambda^2}{4\pi^2 d^2 L}$$

where P_t represents the transmit power (watts), λ is the wavelength of radiation (m), and L is the system loss factor which is independent of propagation environment. The system loss factor represents overall attenuation or loss in the actual system hardware, including transmission line, filter, and antennas. In general, $L > 1$, but $L = 1$ if we assume that there is no loss in the system hardware. It is obvious from above equation that the received power attenuates exponentially with the distance d. The free-space path loss, $PL_F(d)$, without any system loss can be directly derived from the above equation with $L = 1$ as

$$PL_F(d)[dB] = 10log\left(\frac{P_t}{P_r}\right) = -10log\left(\frac{G_t G_r \lambda^2}{4\pi^2 d^2}\right)$$

Without antenna gains (i.e., $G_t = G_r = 1$), above equation is reduced to

$$PL_F(d)[dB] = 10log\left(\frac{P_t}{P_r}\right) = 20log\left(\frac{4\pi d}{\lambda}\right)$$

The aforementioned free-space model, the average received signal in all the other actual environments decreases with the distance between the transmitter and receiver, d, in a logarithmic manner. In fact, a more generalized form of the path loss model can be constructed by modifying the free-space path loss with the path loss exponent n that varies with the environments. This is known as the log-distance path loss model, in which the path loss at distance d is given as

$$PL_{LD}(d)[dB] = PL_F(d_0) + 10nlog\left(\frac{d}{d_0}\right)$$

where d_0 is a reference distance at which or closer to the path loss inherits the characteristics of free-space loss. As shown in Table 1, the path loss exponent can vary from 2 to 6, depending on the propagation environment. Note that $n = 2$ corresponds to the free space. Moreover, n tends to increase as there are more obstructions. Meanwhile, the reference distance d0 must be properly determined for different propagation environments. For example, d_0 is typically set as 1 km for a cellular system with a large coverage (e.g., a cellular system with a cell radius greater than 10 km). However, it could be 100 m or 1 m, respectively, for a macrocellular system with a cell radius of 1km or a microcellular system with an extremely small radius [2].
A log-normal shadowing model is useful when dealing with a more realistic situation. Let X_σ denote a Gaussian random variable with a zero mean and a standard deviation of s. Then, the log-normal shadowing model is given as

$$PL(d)[dB] = \bar{P}L(d) + X_\sigma = PL_F(d_0)10nlog\left(\frac{d}{d_0}\right) + X_\sigma$$

4 Matlab Code

```
% plot_PL_general.m
clear, clf
clc
fc=1.5e9; d0=100; sigma=3; distance=[1:2:31].^2;
Gt=[1 1 0.5]; Gr=[1 0.5 0.5]; Exp=[2 3 6];
for k=1:3
    y_Free(k,:)=PL_free(fc,distance,Gt(k),Gr(k));

y_logdist(k,:)=PL_logdist_or_norm(fc,distance,d0,Exp(k));

y_lognorm(k,:)=PL_logdist_or_norm(fc,distance,d0,Exp(1),sig
ma);
end
figure(1), semilogx(distance,y_Free(1,:),'k-o',
distance,y_Free(2,:),'k-^', distance,y_Free(3,:),'k-'),
grid on, axis([1 1000 40 110]),
title(['Free Path-loss Model, f_c=',num2str(fc/1e6),'MHz'])
xlabel('Distance[m]'), ylabel('Path loss[dB]')
legend('G_t=1, G_r=1','G_t=1, G_r=0.5','G_t=0.5, G_r=0.5')
figure(2)
semilogx(distance,y_logdist(1,:),'k-o',
distance,y_logdist(2,:),'k-^',distance,y_logdist(3,:),'k-
s'), grid on, axis([1 1000 40 110]),
title(['Log-distance Path-loss Model,
f_c=',num2str(fc/1e6),'MHz'])
xlabel('Distance[m]'), ylabel('Path loss[dB]'),
legend('n=2','n=3','n=6')
figure(3), semilogx(distance,y_lognorm(1,:),'k-o',
distance,y_lognorm(2,:),'k-^', distance,y_lognorm(3,:),'k-
s')
grid on, axis([1 1000 40 110]), legend('path 1','path
2','path 3')
title(['Log-normal Path-loss Model,
f_c=',num2str(fc/1e6),'MHz,', '\sigma=', num2str(sigma),
'dB, n=2'])
xlabel('Distance[m]'), ylabel('Path loss[dB]')



function PL=PL_free(fc,d,Gt,Gr)
% Free Space Path Loss Model
% Inputs: fc : Carrier frequency[Hz]
% d : Distance between base station and mobile station[m]
```

```
% Gt/Gr : Transmitter/Receiver gain
% Output: PL : Path loss[dB]
lamda = 3e8/fc; tmp = lamda./(4*pi*d);
if nargin>2, tmp = tmp*sqrt(Gt); end
if nargin>3, tmp = tmp*sqrt(Gr); end
PL = -20*log10(tmp); % )


function PL=PL_logdist_or_norm(fc,d,d0,n,sigma)
% Log-distance or Log-normal shadowing path loss model
% Inputs: fc : Carrier frequency[Hz]
% d : Distance between base station and mobile station[m]
% d0 : Reference distance[m]
% n : Path loss exponent
% sigma : Variance[dB]
lamda=3e8/fc; PL= -
20*log10(lamda/(4*pi*d0))+10*n*log10(d/d0); % Eq.(1.4)
if nargin>4, PL = PL + sigma*randn(size(d));
    end
```

Table 1: Path loss exponent [3].

Environment	Path loss exponent (n)
Free space	2
Urban area cellular radio	2.7–3.5
Shadowed urban cellular radio	3–5
In building line-of-sight	1.6–1.8
Obstructed in building	4–6
Obstructed in factories	2-3

5 Results and Discussion

a. Large Scale Fading

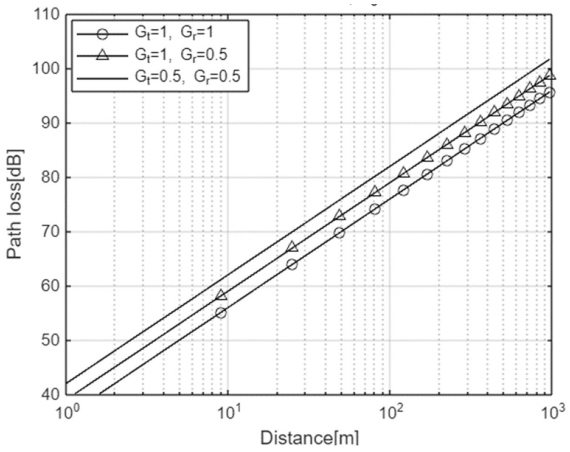


Figure 1: Free Path-Loss Model

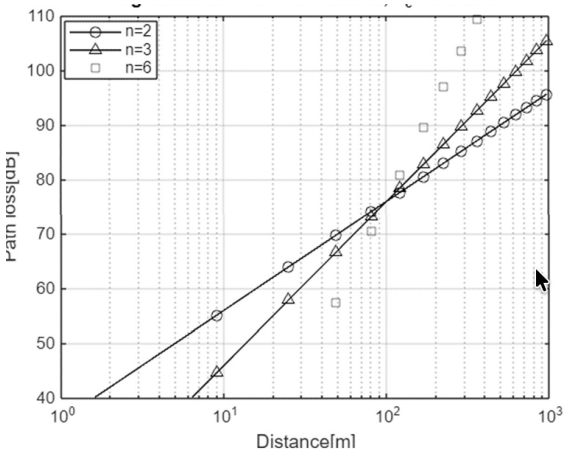


Figure 2: Log-distance Path-Loss Model

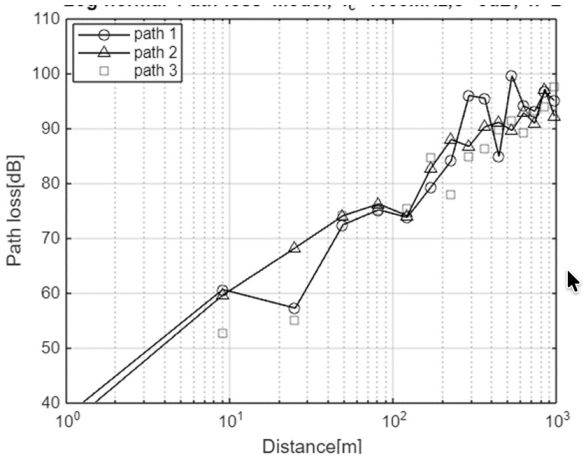


Figure 3: Log-normal Path-Loss Model

b. Small Scale Fading

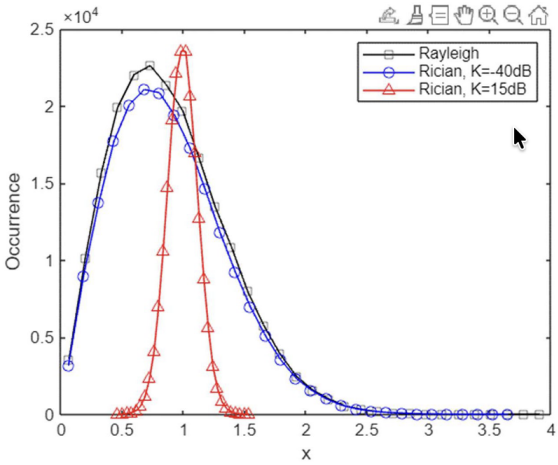


Figure 4: Rayleigh and Rician Fading Channels

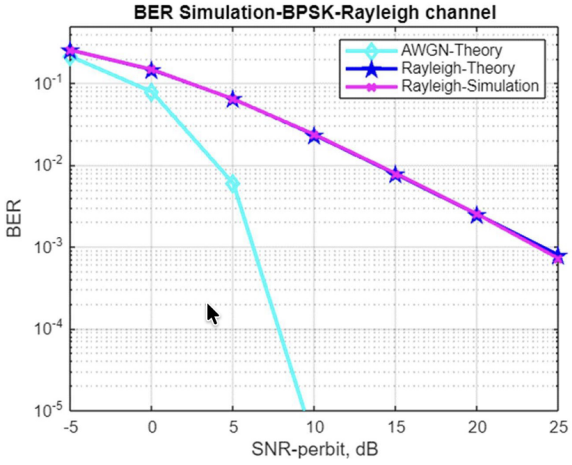


Figure 5: BER of Rayleigh Fading Channel

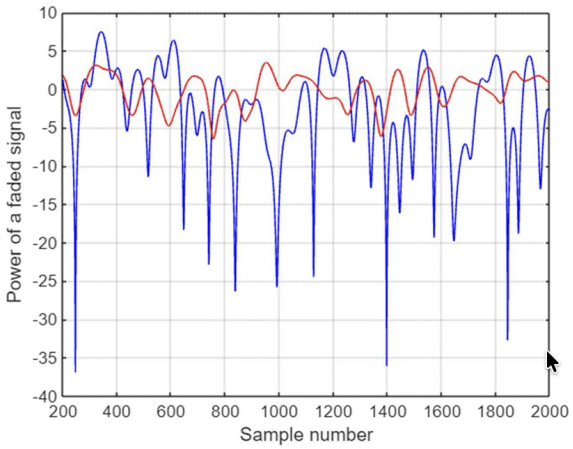


Figure 6: BER of Rayleigh and Rician Fading Channels

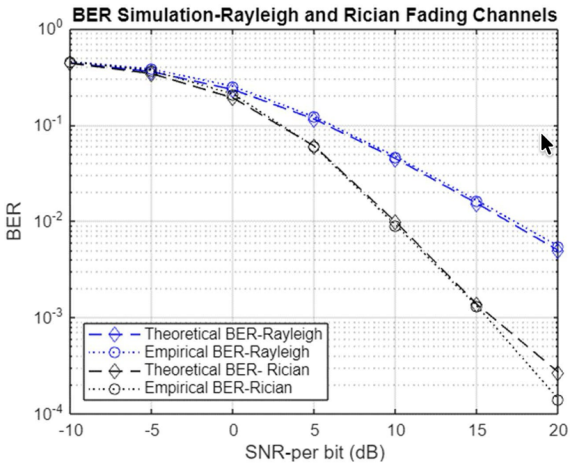


Figure 7: BER Simulatiion- Rayleigh and Rician Fading Channels

c. Link Budgets

Link Budget

Tag	Name	L1	L2	L3
N1	Distance (km)	3.7865e+03	4.0215e+04	3.6138e+04
N2	Elevation (deg)	18.3253	13.5231	41.5653
N3	Tx EIRP (dBW)	32	46	47
N4	Polarization loss (dB)	3.0103	3.0103	3.0103
N5	FSPL (dB)	186.9352	205.3634	206.5296
N6	Received isotropic power (dBW)	-160.9455	-165.3737	-165.5399
N7	C/No (dB-Hz)	90.6537	86.2255	86.0592
N8	C/N (dB)	22.8722	18.4439	18.2777
N9	Received Eb/No (dB)	20.6537	16.2255	16.0592
N10	Margin (dB)	8.6537	4.2255	4.0592

Figure 8: Link Budget

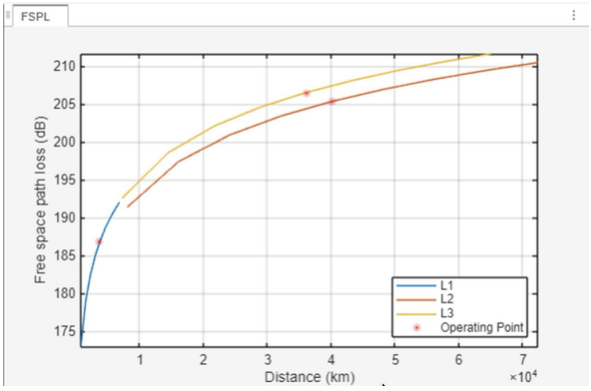


Figure 9: FSPL

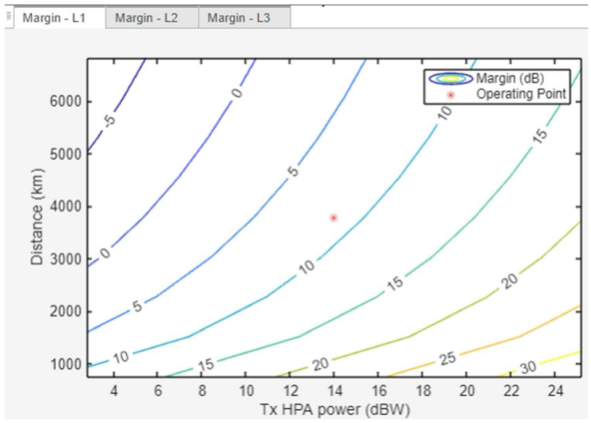


Figure 10: Margin - L1

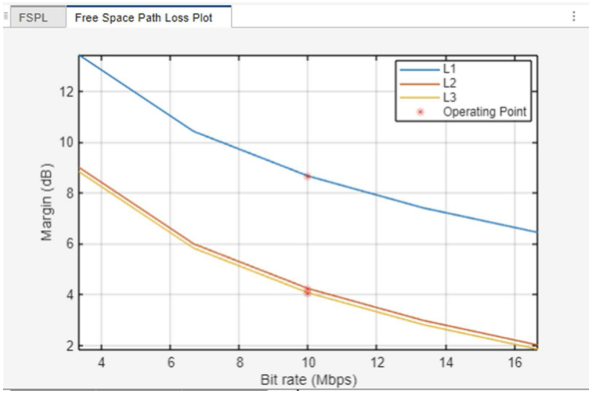


Figure 11: FSPL PLot

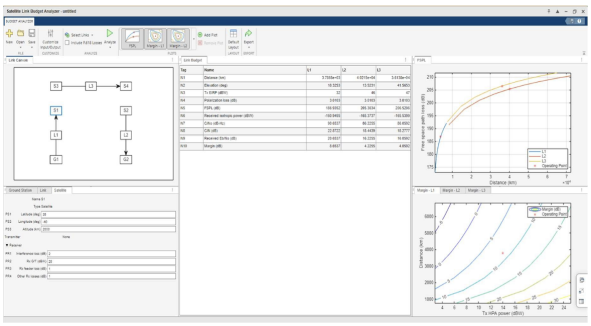


Figure 12: Simulation of Link Budget Analyzer

References

[1] H. Friis, "A note on a simple transmission formula." in *proc. IRE*, vol. 35, no. 5, 1946, pp. 254—-256.

[2] W. Lee, "Mobile communications engineering," 1985.

[3] T. Rappaport, "Wireless communications: Principles and practice 2/e," 2001.

Study of Receive Diversity Concepts-Selection diversity, Maximum ratio combining, and Equal gain diversity

Date Performed:
Exp. No.:
Reg. No.:
Name:

1 Objective

To study the diversity concepts with the aid of simulations based on MATLAB or IT++ library.

2 Tool(s) Required

MATLAB OR IT++ LIBRARY

3 Theory

Space diversity reception methods can be classified into four categories.

a. Selection diversity

b. Feedback diversity

c. Maximal ratio combining

d. Equal gain diversity

3.1 Selection diversity

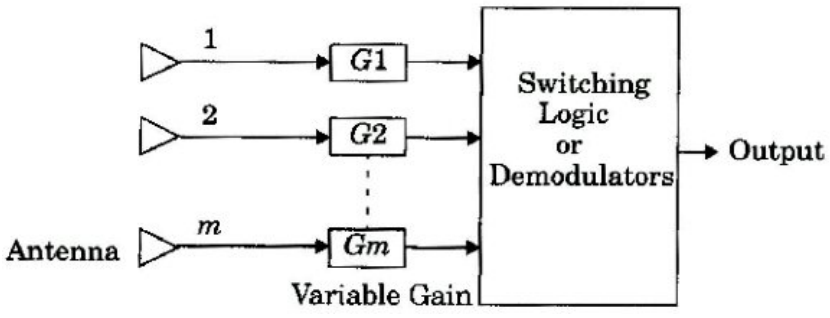


Figure 1: Generalized block diagram of space diversity [1].

Selection diversity is the simplest diversity technique. A block diagram of this method is similar to the one shown in Fig.1. Here, m demodulators are used to provide m diversity branches whose gains are adjusted to provide the same average SNR for each branch. The receiver branch having the highest instantaneous SNR is connected to the demodulator. The antenna signals themselves could be sampled and the best one sent to a single demodulator. In practice, the branch with the largest $(S+N)/N$ is used, since it is difficult to measure SNR. A practical selection diversity system cannot function on a truly instantaneous basis, but must be designed so that the internal time constants of the selection circuitry are shorter than the reciprocal of the signal fading rate.

3.2 Maximal ratio combining (MRC)

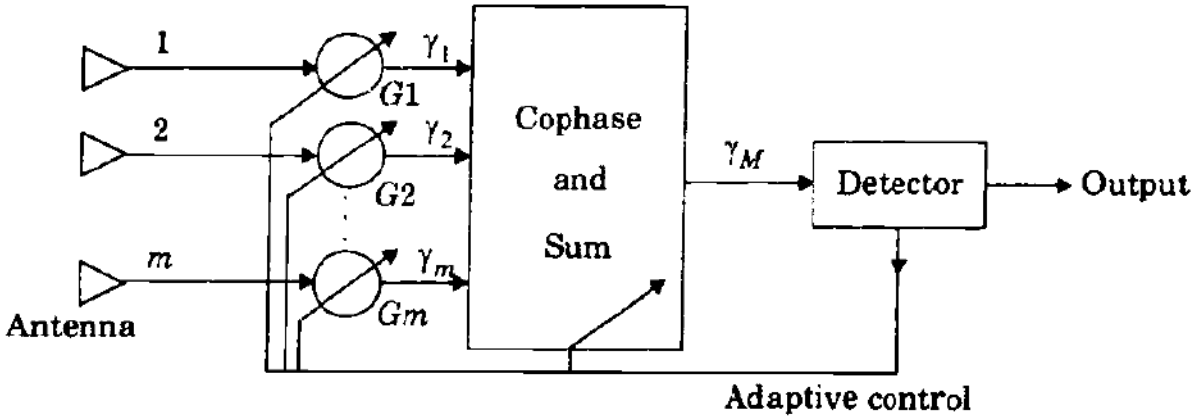


Figure 2: Maximal ratio combiner [1].

In this method, the signals from all of the M branches are weighted according to their individual signal voltage to noise power ratios and then summed. Fig.2 shows a block diagram of the technique. Here, the individual signals must be co-phased before being summed (unlike selection diversity), which generally requires an individual receiver and phasing circuit for each antenna element. Maximal ratio combining produces an output SNR equal to the sum of the individual SNRs. Thus, it has the advantage of producing an output with an acceptable SNR even when none of the individual signals are themselves acceptable. This technique gives the best statistical reduction of fading of any known linear diversity combiner. Modern DSP techniques and digital receivers are now making this optimal form of diversity practical.

3.3 Equal gain diversity (EGC)

In certain cases, it is not convenient to provide for the variable weighting capability required for true maximal ratio combining. In such cases, the branch weights are all set to unity but the signals from each branch are co-phased to provide equal gain combining diversity. This allows the receiver to exploit signals that are simultaneously received on each branch. The possibility of producing an acceptable signal from a number of unacceptable inputs is still retained.

4 Matlab Code

```
clc;
clear all;
close all;

sample_num = 100000;

% Preallocation for Bit error rate
Pe_sc = zeros(5, 4, 2); % selective combining
Pe_mrc = zeros(5,4, 2); % maximal ratio combining
Pe_egc = zeros(5,4, 2); % equal gain combining
tic


for R = 0:1 % Rayleigh:0 ; Ricean:1
    % Different fading channel has different variance
    if R == 0
        sigma = 1/sqrt(2);
    else
        sigma = 1/2;
    end
    SNR=-10:2:12;
    for enr_index = 1:length(SNR)
        %enr_dB = (enr_index-1)*2+1; %
        %enr = 10^(enr_dB/10);
        enr = 10^(enr_index/10);

        % generate QPSK data (2 bits)
        data = rand(2,sample_num); % 2*sample_num matrix
        data = 2*(data > 0.5)-1; % map to -1, 1

        Edata = sqrt(2); % symbol energy
        En = Edata/enr; % noise energy

        for L = 1:4
            % generate noise
            n = normrnd(0,sqrt(En/2),2,sample_num,L) +
1i*normrnd(0,sqrt(En/2),2,sample_num,L);

            % generate fading gain (R=0: Rayleigh; R=1:
Riciean)
            g = normrnd(R/2,sigma,1,sample_num,L) +
1i*normrnd(R/sqrt(2),1/2,1,sample_num,L);
            g_tmp = repmat(g,2,1,1); % replicate for 2 bits
(QPSK)
```

```
            tx_data = repmat(data,1,1,L); % replicate for L
branches
            % received signal
            r = g_tmp.*tx_data + n;

            %%% Selective combining %%%
            [Pe_sc(enr_index, L, R+1), result_sc] =
selective_combining(g, g_tmp, r, sample_num, data);

            %%% Maximal Ratio Combining %%%
            [Pe_mrc(enr_index, L, R+1), result_mrc] =
maximal_ratio_combining(g_tmp, r, sample_num, data);

            %%% Equal Gain Combining %%%
            [Pe_egc(enr_index, L, R+1), result_egc] =
equal_gain_combining(g_tmp, r, sample_num, data);

          end
    end
end
toc



% Plot Rayleigh
figure,plot(SNR, Pe_sc(:,:,1),'-*')
set(gca, 'YScale', 'log')
title('BER of Selective Combining (Rayleigh)');
legend('L=1','L=2','L=3','L=4');
xlabel('SNR (dB)');
ylabel('Bit error rate');



figure,plot(SNR, Pe_mrc(:,:,1),'-*')
set(gca, 'YScale', 'log')
title('BER of Maximal Ratio Combining (Rayleigh)');
legend('L=1','L=2','L=3','L=4');
xlabel('SNR (dB)');
ylabel('Bit error rate');
```

```
figure,plot(SNR, Pe_egc(:,:,1),'-*')
set(gca, 'YScale', 'log')
title('BER of Equal Gain Combining (Rayleigh)');
legend('L=1','L=2','L=3','L=4');
xlabel('SNR (dB)');
ylabel('Bit error rate');


% Plot Riciean

figure,plot(SNR, Pe_sc(:,:,2),'-*')
set(gca, 'YScale', 'log')
title('BER of Selective Combining (Ricean)');
legend('L=1','L=2','L=3','L=4');
xlabel('SNR (dB)');
ylabel('Bit error rate');


figure,plot(SNR, Pe_mrc(:,:,2),'-*')
set(gca, 'YScale', 'log')
title('BER of Maximal Ratio Combining (Ricean)');
legend('L=1','L=2','L=3','L=4');
xlabel('SNR (dB)');
ylabel('Bit error rate');

figure,plot(SNR, Pe_egc(:,:,2),'-*')
set(gca, 'YScale', 'log')
title('BER of Equal Gain Combining (Ricean)');
legend('L=1','L=2','L=3','L=4');
xlabel('SNR (dB)');
ylabel('Bit error rate');

% Comparison among all combining strategies
% Rayleigh

figure,plot(SNR, Pe_sc(:,4,1),'-+',SNR, Pe_egc(:,4,1),'-
*',SNR, Pe_mrc(:,4,1),'-+')
set(gca, 'YScale', 'log')
title('BER of SC & MRC & EGC (Rayleigh)');
legend('Selective Combining','Equal Gain Combinig','Maximal
Ratio Combining');
xlabel('SNR (dB)');
ylabel('Bit error rate');
```

```
% Ricean

figure,plot(SNR, Pe_sc(:,4,2),'-+',SNR, Pe_egc(:,4,2),'-
*',SNR, Pe_mrc(:,4,2),'-+')

set(gca, 'YScale', 'log')
title('BER of SC & MRC & EGC (Ricean)');
legend('Selective Combining','Equal Gain Combinig','Maximal
Ratio Combining');
xlabel('SNR (dB)');
ylabel('Bit error rate');



function [BER, result] = selective_combining(g, g_2, r,
sample_num, data)
% Selective Combining:
% First, find the max branch index in g(1*sample_num*branch
number) matrix.
% Second, get corresponding gain and received signal and
detect its symbol.
% Third, calculate the BER.

g_abs = abs(g); % energy of fading gain
[~, I] = max(g_abs,[],3); % find index of fading gain
r_sc_tmp = zeros(2,sample_num);
g_sc = zeros(2,sample_num);

% Select corresponding fading gain and received signal from
indices
for i = 1:sample_num
   r_sc_tmp(:,i) = r(:,i,I(i));
   g_sc(:,i) = g_2(:,i,I(i));
end
r_sc = real(exp(-1i*angle(g_sc)).*r_sc_tmp); % remove the
phase shift
result = (r_sc > 0)*2 -1; % map to +1, -1

% Calculate bit error rate
BER = get_BER(result, data, sample_num);
```

```
function [BER, result] = maximal_ratio_combining(g_2, r,
sample_num, data)
% Maximal ratio combining

r_mrc = real(sum(conj(g_2).*r,3)); % weighted by fading
gain and combine
result = (r_mrc > 0)*2 -1; % Detection
BER = get_BER(result, data, sample_num);


function [BER, result] = equal_gain_combining(g_2, r,
sample_num, data)
% Equal gain combining

r_egc = real(sum(exp(-1i*angle(g_2)).*r,3)); % Cancel phase
shift and combine
result = (r_egc > 0)*2 -1; % Detection
BER = get_BER(result, data, sample_num);


function Pe = get_BER(A,B,sample_num)
% Calculate bit error rate by counting how many elements in
two matrices A
% and B are different. Next, divided the size the matrix to
derive BER.
wrong = A~= B;
error_count = sum(sum(wrong));
Pe = error_count/sample_num/2;
```

5 Results and Discussion

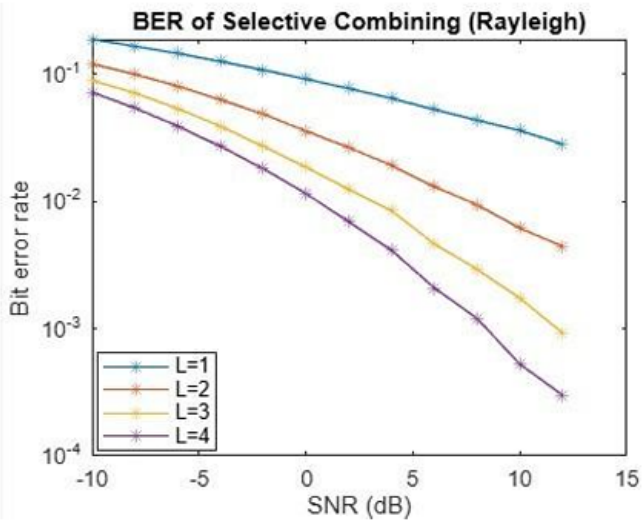


Figure 3: BER of Selective Combining (Rayleigh)

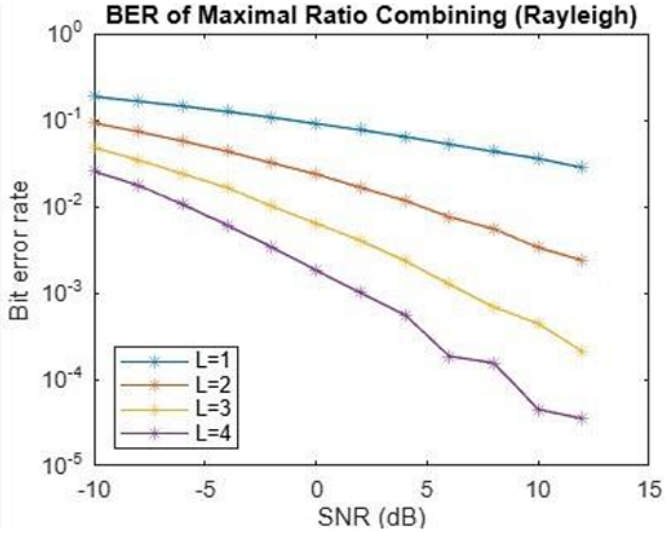


Figure 4: BER of Maximal Ratio Combining (Rayleigh)

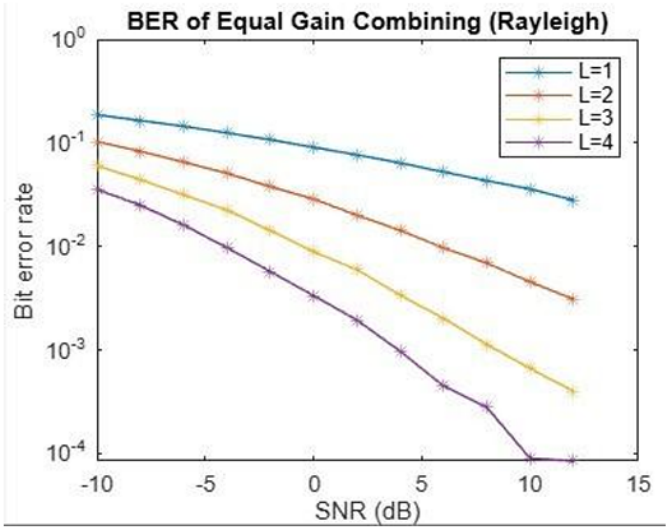


Figure 5: BER of Equal Gain Combining (Rayleigh)

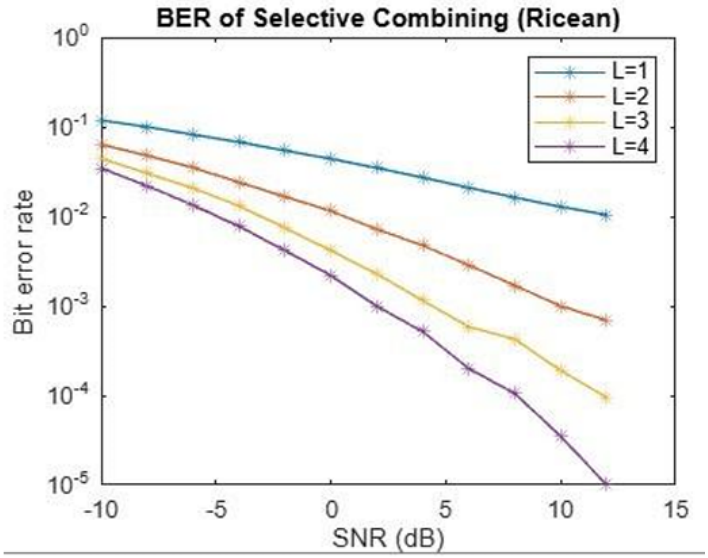


Figure 6: BER of Selective Combining (Rician)

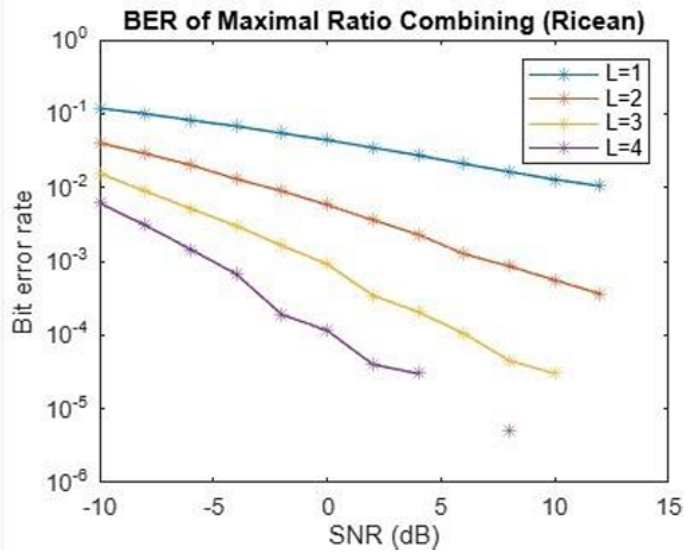


Figure 7: BER of Maximal Ratio Combining (Rician)

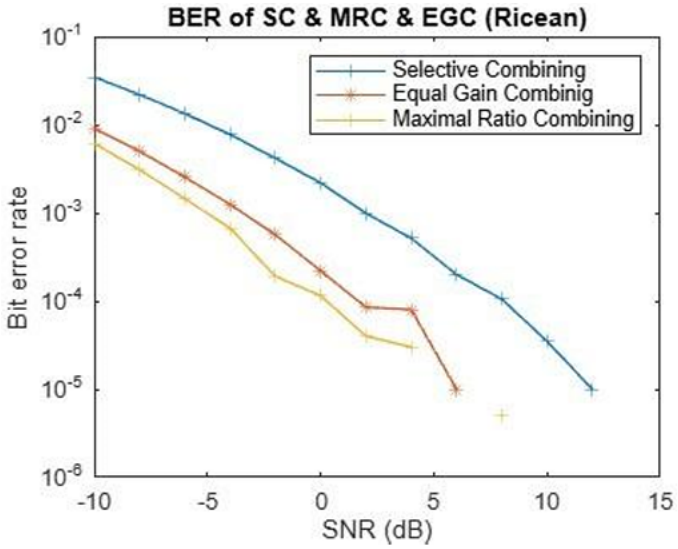


Figure 8: BER of Equal Gain Combining (Rician)

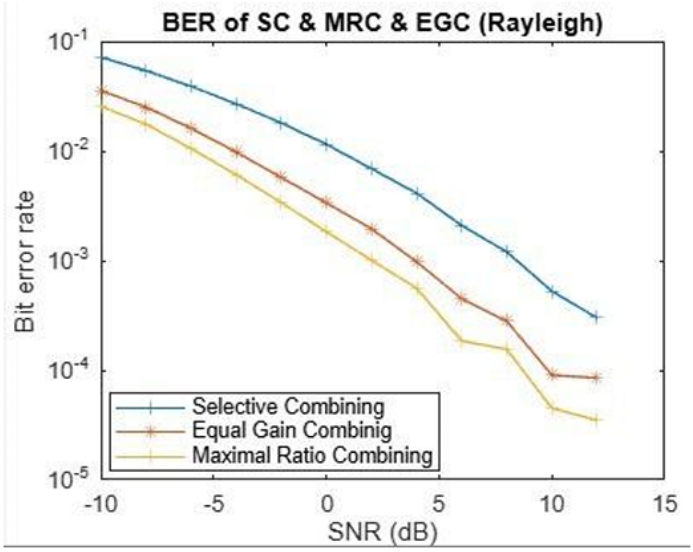


Figure 9: BER of SC, MRC and EGC (Rayleigh)

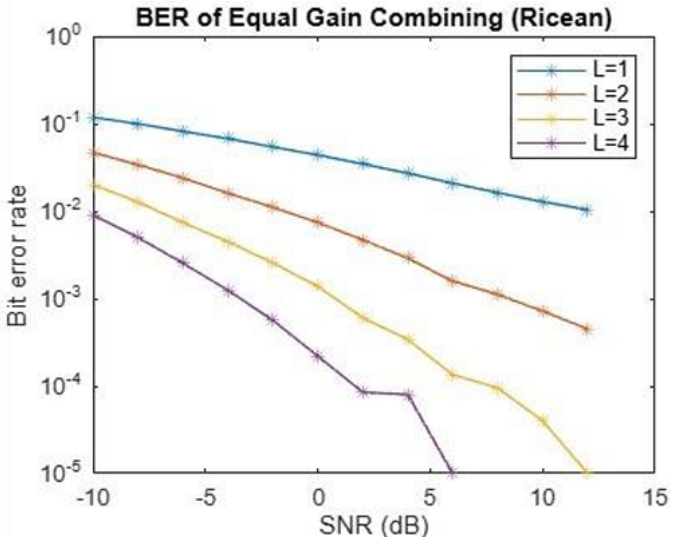


Figure 10: BER of SC, MRC and EGC (Rician)

References

[1] T. S. Rappaport, *Wireless Communications: Principles and Practice*. Pearson, 2010.

Study of diversity Concepts-Transmit Diversity

Date Performed:
Exp. No.:
Reg. No.:
Name:

1 Objective

To study Alamouti transmit diversity with the aid of simulations based on MATLAB or IT++ library.

2 Tool(s) Required

MATLAB OR IT++ LIBRARY

3 Theory

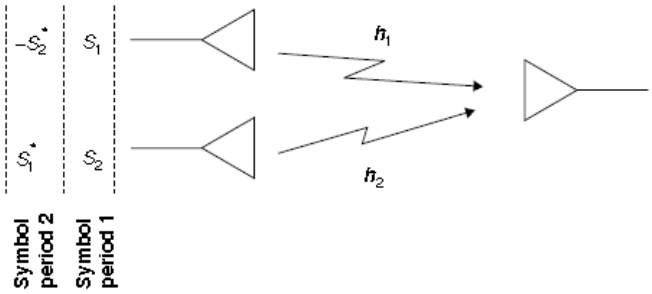


Figure 1: A schematic of the transmission strategy in the Alamouti scheme

Exploiting spatial diversity in systems with multiple antennas at the transmitter requires that the signal be pre-processed or pre-coded prior to transmission. There has been increased interest in these techniques since the 1990s. The need for pre-processing the transmit signal is explained below. Consider a symbol s that is to be transmitted over a system with two transmit antennas and a single receive antenna. A simplistic attempt to exploit diversity would be to transmit the signal from both transmit antennas at the same time. Assuming a flat fading environment, where the channel signatures corresponding to the transmit antennas are given by h_1 and h_2, the received signal y may be expressed as

$$y = \sqrt{\frac{E_s}{2}}(h_1 + h_2)s + n \tag{1}$$

where E_s is the average energy available at the transmitter over a symbol period evenly divided between the transmit antennas and n is the zero mean circularly symmetric complex Gaussian (ZMCSCG) noise at the receiver. Noting that the sum of two complex Gaussian random variables is also complex Gaussian, $\frac{1}{\sqrt{2}}(h_1+h_2)$ is ZMCSCG with unit variance. As a result, Eq. (1) may be expressed as

$$y = \sqrt{E_s}(h)s + n \tag{2}$$

where h is ZMCSCG with $E[|h|^2] = 1$. Therefore, this naive technique does not provide diversity.
Let us now, consider a simple but ingenious transmit diversity technique – the Alamouti scheme. In this technique, two different symbols s_1 and s_2 are transmitted simultaneously from antennas 1 and 2 respectively during the first symbol period, followed by signals $-s_2^*$ and s_1^* from antennas 1 and , respectively, during the next symbol period (Fig. 1). We assume that the channel remains constant over the two symbol periods, and is frequency flat. Therefore, $\mathbf{h} = [h_1, h_2]$ and the signals y_1 and y_2 received over the two symbol periods are given by

$$y_1 = \sqrt{\frac{E_s}{2}}h_1 s_1 + \sqrt{\frac{E_s}{2}}h_2 s_2 + n_1 \tag{3}$$

$$y_2 = -\sqrt{\frac{E_s}{2}}h_1 s_2^* + \sqrt{\frac{E_s}{2}}h_2 s_1^* + n_2 \tag{4}$$

where n_1 and n_2 are ZMCSCG noise with $E[|n_1|^2] = E[|n_2|^2] = N_o$ and $E_s/2$ is the average transmit energy per symbol period per antenna. The receiver forms a rearranged signal vector y as follows:

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2^* \end{bmatrix} \tag{5}$$

The vector $\mathbf{y}$ can be expressed as

$$\mathbf{y} = \sqrt{\frac{E_s}{2}}\begin{bmatrix} h_1 & h_2 \\ h_2^* & -h_1* \end{bmatrix}\begin{bmatrix} s_1 \\ s_2 \end{bmatrix} + \begin{bmatrix} n_1 \\ n_2^* \end{bmatrix} = \sqrt{\frac{E_s}{2}}\mathbf{H}_{eff}\mathbf{S} + \mathbf{n} \tag{6}$$

where $\mathbf{s} = [s_1 s_2]^T$ and $\mathbf{n} = [n_1 n_2^*]^T$. The effective channel matrix $\mathbf{H}_{eff}$ is orthogonal.

4 Matlab Code

In this section, we present the MATLAB codes that in turn can be used to simulate the transmission strategy in the Alamouti scheme.
Refer to [1], [2], [3], [4], and [5],for additional information about synchronization techniques.

```
clear all;
close all;
clc;


snapshots = 100000;
EbNo = 0:10;
K = [4.0; 0.6];
M = [4; 8]; %Positions of modulation (M-PSK)
Mt = 2;
Mr = [1; 2];
ostbcEnc = comm.OSTBCEncoder('NumTransmitAntennas', Mt); % Alamouti
ric_ber = zeros(length(EbNo), length(M), length(K), length(Mr));
sum_BER = zeros(length(EbNo), length(M), length(K), length(Mr));
for mr = 1:length(Mr)
    ostbcComb = comm.OSTBCCombiner('NumTransmitAntennas', Mt,
'NumReceiveAntennas', Mr(mr));
    H = zeros(Mr(mr), Mt, snapshots);
    ric_msg = zeros(snapshots, Mr(mr));
    for k = 1:length(K)
        mu = sqrt( K(k)/(K(k)+1));
        s = sqrt(1/(K(k)+1));
        for m = 1:length(M)
            hModulator = comm.PSKModulator('ModulationOrder', M(m),
'BitInput', false);
            hDemod = comm.PSKDemodulator('ModulationOrder', M(m),
'BitOutput', false);
            ric_ber(:,m,k,mr) = berfading(EbNo, 'psk', M(m),
Mr(mr)*Mt, K(k));
            snr = EbNo+10*log10(log2(M(m)));
            message = randi([0,M(m)-1],100000,1);
            mod_msg = step(hModulator,message);
            Es = mean(abs(mod_msg).^2);
            alam_msg = step(ostbcEnc, mod_msg);
            % Channel
            h = mu + s*(1/sqrt(2))*(randn(Mr(mr),Mt,snapshots/Mt)...
            + 1j*randn(Mr(mr),Mt, snapshots/Mt));
            H(:,:,1:2:end-1) = h;
            H(:,:,2:2:end) = h;
            pathGainself = permute(H,[3,2,1]);
            % Transmit through the channel
            for q = 1:snapshots;
                ric_msg(q,:) =
(sqrt(Es/Mt)*H(:,:,q)*alam_msg(q,:).').';
            end
            for c = 1:100
                for jj = 1:length(EbNo)
                    noisy_mod = awgn(ric_msg,snr(jj),'measured','dB');
                    decodeData =
step(ostbcComb,noisy_mod,pathGainself);
                    demod_msg = step(hDemod,decodeData);
```

```
                    [number,BER(c,jj)] = biterr(message,demod_msg);
                end
            end
            sum_BER(:,m, k, mr) = sum(BER)./c;
        end
    end
end
figure(1)
semilogy(EbNo,sum_BER(:,1,1,1),'r-o',EbNo,sum_BER(:,2,1,1),'g-o',...
         EbNo,ric_ber(:,1,1,1),'r-',EbNo,ric_ber(:,2,1,1),'g-',...
         EbNo,sum_BER(:,1,1,2),'b-o',EbNo,sum_BER(:,2,1,2),'y-o',...
         EbNo,ric_ber(:,1,1,2),'b-',EbNo,ric_ber(:,2,1,2),'y-',...
         'LineWidth', 1.5)
title('Rician model (K = 4.0)')
legend('QPSK(simulated) 2x1', '8-PSK(simulated) 2x1',...
    'QPSK(theory) 2x1','8-PSK(theory) 2x1',...
    'QPSK(simulated) 2x2', '8-PSK(simulated) 2x2',...
    'QPSK(theory) 2x2','8-PSK(theory) 2x2')
xlabel('EbNo (dB)')
ylabel('BER')
grid on
figure(2)
semilogy(EbNo,sum_BER(:,1,2,1),'r-o',EbNo,sum_BER(:,2,2,1),'g-o',...
         EbNo,ric_ber(:,1,2,1),'r-',EbNo,ric_ber(:,2,2,1),'g-',...
         EbNo,sum_BER(:,1,2,2),'b-o',EbNo,sum_BER(:,2,2,2),'y-o',...
         EbNo,ric_ber(:,1,2,2),'b-',EbNo,ric_ber(:,2,2,2),'y-',...
         'LineWidth', 1.5)
title('Rician model (K = 0.6)')
legend('QPSK(simulated) 2x1', '8-PSK(simulated) 2x1',...
    'QPSK(theory) 2x1','8-PSK(theory) 2x1',...
    'QPSK(simulated) 2x2', '8-PSK(simulated) 2x2',...
    'QPSK(theory) 2x2','8-PSK(theory) 2x2')
xlabel('EbNo (dB)')
ylabel('BER')
grid on
```

5 Results and Discussion

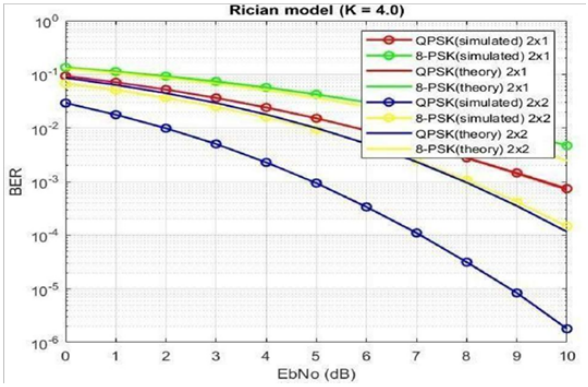


Figure 2: Rician model for k=4.0

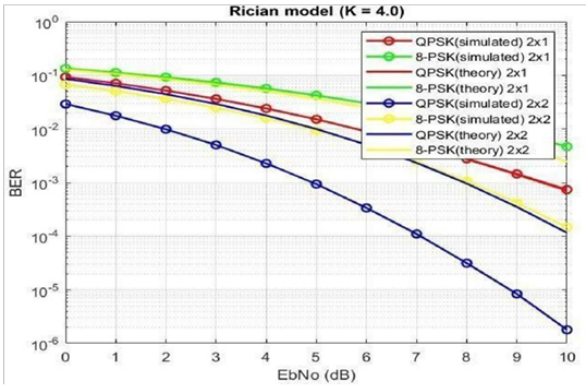


Figure 3: Rician model for k=0.6

References

[1] C. Oestges and B. Clerckx, *MIMO Wireless Communications: From Real-World Propagation to Space-Time Code Design,.* CRC Press, Boca Raton, FL, 2006.

[2] E. George Tsoulos, *MIMO System Technology for Wireless Communications,.* Academic Press, 2007.

[3] S. M. Alamouti, "A simple transmit diversity technique for wireless communications," *IEEE Journal on Selected Areas in Communications*, vol. 16, no. 8, pp. 1451–1458, Oct, 1998.

[4] M. Jankiraman, *Space-time codes and MIMO systems,.* Artech House, Boston, 2004.

[5] V. Tarokh, N. Seshadri, and A. R. Calderbank, "A simple transmit diversity technique for wireless communications," *IEEE Transactions on Information Theory*, vol. 44, no. 2, pp. 744–765, Mar., 1998.

Simulation of MIMO system: Spatial Multiplexing and Interference Cancellation

Date Performed:
Exp. No.:
Reg. No.:
Name:

1 Objective

To study spatial multiplexing and interference cancellation techniques in a MIMO system with the aid of simulations based on MATLAB or IT++ library.

2 Tool(s) Required

MATLAB OR IT++ LIBRARY

3 Theory

This simulation experiment shows spatial multiplexing schemes wherein the data stream is subdivided into independent sub-streams, one for each transmit antenna employed. As a consequence, these schemes provide a multiplexing gain and do not require explicit orthogonalization as needed for space-time block coding.
Spatial multiplexing requires powerful decoding techniques at the receiver though. Of the many proposed [1], this example highlights two ordered Successive Interference Cancellation (SIC) detection schemes. These schemes are similar to the original Bell Labs Layered Space-Time (BLAST) techniques as per [2], [3].
For expositional benefits the example uses the basic 2x2 MIMO system employing two transmit and two receive antennas. For an uncoded QPSK modulated system it employs flat Rayleigh fading over independent transmit-receive links. At the receiver end, we assume perfect channel knowledge with no feedback to the transmitter, i.e., an open-loop spatial multiplexing system.
The experiment shows two nonlinear interference cancellation methods - Zero-Forcing (ZF) and Minimum-Mean-Square-Error (MMSE) [4] - with symbol cancellation and compares their performance with the Maximum-Likelihood (ML) optimum receiver.

4 Matlab Code

In this section, we present the MATLAB codes that in turn can be used to simulate spatial multiplexing and interference cancellation techniques.
Refer to [1]-[5], for additional information on spatial multiplexing and interference cancellation.

```
clc
clear all;

N = 2;                      % Number of transmit antennas
M = 2;                      % Number of receive antennas
EbNoVec = -10:5:20;         % Eb/No in dB
modOrd = 2;                 % constellation size = 2^modOrd

% Create a local random stream to be used by random number
generators for
% repeatability.
stream = RandStream('mt19937ar');

% Create PSK modulator and demodulator System objects
pskModulator   = comm.PSKModulator(...
            'ModulationOrder',  2^modOrd, ...
            'PhaseOffset',      0, ...
            'BitInput',         true);
pskDemodulator = comm.PSKDemodulator( ...
            'ModulationOrder',  2^modOrd, ...
            'PhaseOffset',      0, ...
            'BitOutput',        true);

% Create error rate calculation System objects for 3
different receivers
zfBERCalc = comm.ErrorRate;
mmseBERCalc = comm.ErrorRate;
mlBERCalc = comm.ErrorRate;

% Get all bit and symbol combinations for ML receiver
allBits = de2bi(0:2^(modOrd*N)-1, 'left-msb')';
allTxSig = reshape(pskModulator(allBits(:)), N,
2^(modOrd*N));

% Pre-allocate variables to store BER results for speed
[BER_ZF, BER_MMSE, BER_ML] = deal(zeros(length(EbNoVec),
3));

% Set up a figure for visualizing BER results
fig = figure;
grid on;
hold on;
ax = fig.CurrentAxes;
ax.YScale = 'log';
```

```
xlim([EbNoVec(1)-0.01 EbNoVec(end)]);
ylim([1e-3 1]);
xlabel('Eb/No (dB)');
ylabel('BER');
fig.NumberTitle = 'off';
fig.Renderer = 'zbuffer';
fig.Name = 'Spatial Multiplexing';
title('2x2 Uncoded QPSK System');
set(fig,'DefaultLegendAutoUpdate','off');

% Loop over selected EbNo points
for idx = 1:length(EbNoVec)
    % Reset error rate calculation System objects
    reset(zfBERCalc);
    reset(mmseBERCalc);
    reset(mlBERCalc);

    % Calculate SNR from EbNo for each independent
transmission link
    snrIndB = EbNoVec(idx) + 10*log10(modOrd);
    snrLinear = 10^(0.1*snrIndB);

    while (BER_ZF(idx, 3) < 1e5) && ((BER_MMSE(idx, 2) <
100) || ...
          (BER_ZF(idx, 2) < 100) ||  (BER_ML(idx, 2)   <
100))
        % Create random bit vector to modulate
        msg = randi(stream, [0 1], [N*modOrd, 1]);

        % Modulate data
        txSig = pskModulator(msg);

        % Flat Rayleigh fading channel with independent
links
        rayleighChan = (randn(stream, M, N) +
1i*randn(stream, M, N))/sqrt(2);

        % Add noise to faded data
        rxSig = awgn(rayleighChan*txSig, snrIndB, 0,
stream);

        % ZF-SIC receiver
        r = rxSig;
```

```
        H = rayleighChan; % Assume perfect channel
estimation
        % Initialization
        estZF = zeros(N*modOrd, 1);
        orderVec = 1:N;
        k = N+1;
        % Start ZF nulling loop
        for n = 1:N
            % Shrink H to remove the effect of the last
decoded symbol
            H = H(:, [1:k-1,k+1:end]);
            % Shrink order vector correspondingly
            orderVec = orderVec(1, [1:k-1,k+1:end]);
            % Select the next symbol to be decoded
            G = (H'*H) \ eye(N-n+1); % Same as inv(H'*H),
but faster
            [~, k] = min(diag(G));
            symNum = orderVec(k);

            % Hard decode the selected symbol
            decBits = pskDemodulator(G(k,:) * H' * r);
            estZF(modOrd * (symNum-1) + (1:modOrd)) =
decBits;

            % Subtract the effect of the last decoded
symbol from r
            if n < N
                r = r - H(:, k) * pskModulator(decBits);
            end
        end

        % MMSE-SIC receiver
        r = rxSig;
        H = rayleighChan;
        % Initialization
        estMMSE = zeros(N*modOrd, 1);
        orderVec = 1:N;
        k = N+1;
        % Start MMSE nulling loop
        for n = 1:N
            H = H(:, [1:k-1,k+1:end]);
            orderVec = orderVec(1, [1:k-1,k+1:end]);
            % Order algorithm (matrix G calculation) is the
only difference
```

```
            % with the ZF-SIC receiver
            G = (H'*H + ((N-n+1)/snrLinear)*eye(N-n+1)) \
eye(N-n+1);
            [~, k] = min(diag(G));
            symNum = orderVec(k);

            decBits = pskDemodulator(G(k,:) * H' * r);
            estMMSE(modOrd * (symNum-1) + (1:modOrd)) =
decBits;

            if n < N
                r = r - H(:, k) * pskModulator(decBits);
            end
        end

        % ML receiver
        r = rxSig;
        H = rayleighChan;
        [~, k] = min(sum(abs(repmat(r,[1,2^(modOrd*N)]) -
H*allTxSig).^2));
        estML = allBits(:,k);

        % Update BER
        BER_ZF(  idx, :) = zfBERCalc(msg, estZF);
        BER_MMSE(idx, :) = mmseBERCalc(msg, estMMSE);
        BER_ML(  idx, :) = mlBERCalc(msg, estML);
    end

   end

semilogy(EbNoVec, BER_ZF(  :, 1), 'r*-', ...
         EbNoVec, BER_MMSE(:, 1), 'bo-', ...
         EbNoVec, BER_ML(  :, 1), 'gd-');
hold off;
legend('ZF-SIC', 'MMSE-SIC', 'ML');
```

5 Results and Discussion

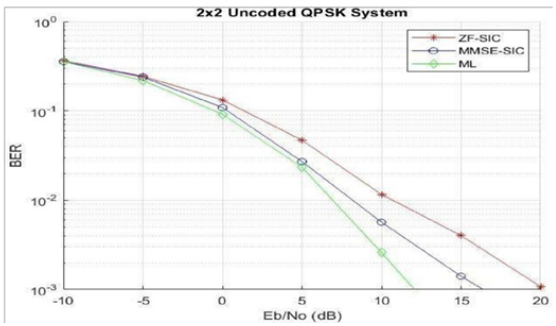


Figure 1: 2x2 uncoded QPSK System

References

[1] E. George Tsoulos, *MIMO System Technology for Wireless Communications.* CRC Press, Boca Raton, FL., 2006.

[2] G. J. Foschini, "Layered space-time architecture for wireless communication in a fading environment when using multiple antennas," *The Bell Sys. Tech. Journal*, no. 1, pp. 41–59, 1996.

[3] P. W. Wolniansky, G. J. Foschini, G. D. Golden, and V. R. A, "V-blast: An architecture for realizing very high data rates over the rich scattering wireless channel," in *proc. 1998 URSI International Symposium on Signals, Systems, and Electronics*, 29 Sep.-2 Oct. 1998, pp. 295–300.

[4] X. Li, H. C. Huang, A. Lozano, and G. J. Foschini, "Reduced-complexity detection algorithms for systems using multi-element arrays," in *proc. IEEE Global Telecommunications Conference*, vol. 2, no. 27, Nov.-1 Dec. 2000, pp. 1072–76.

[5] Y. Shang and X.-G. Xia, "On fast recursive algorithms for v-blast with optimal ordered sic detection," *IEEE Trans. Wireless Communications*, vol. 8, no. 6, pp. 2860–2865, 2009.

Implementation of IEEE 802.11n standard (PHY Layer)

Date Performed:
Exp. No.:
Reg. No.:
Name:

1 Objective

To study the acievable packet error rate of an IEEE 802.11n HT link using an end-to-end simulation with a fading TGn channel model and additive white Gaussian noise with the aid of simulation based on MATLAB or IT++ library.

2 Tool(s) Required

MATLAB OR IT++ LIBRARY

3 Theory

An end-to-end simulation is used to determine the packet error rate for an 802.11n high throughput (HT) [1] link with a fading channel at a selection of SNR points. At each SNR point multiple packets are transmitted through a channel, demodulated and the PSDUs recovered. The PSDUs are compared to those transmitted to determine the number of packet errors and hence the packet error rate. Packet detection, timing synchronization, carrier frequency offset correction and phase tracking are performed by the receiver. The processing for each packet is summarized in the Fig.1.

The HT format configuration object, wlanHTConfig, contains the format specific configuration of the transmission. The properties of the object contain the configuration. In the simulation, the object is configured for a 20 MHz channel bandwidth, 2 transmit antennas, 2 space time streams and no space time block coding. In this experiment a TGn N-LOS channel model is used with delay profile Model-B. For Model-B when the distance between transmitter and receiver is greater than or equal to five meters, the model is NLOS.

For each SNR point in the vector snr a number of packets are generated, passed through a channel and demodulated to determine the packet error rate.The number of packets tested at each SNR point is controlled by two parameters: maxNumPEs is the maximum number of packet errors simulated at each SNR point. When the number of packet errors reaches this limit, the simulation at this SNR point is complete. maxNumPackets is the maximum number of packets simulated at each SNR point and limits the length of the simulation if the packet error limit is not reached. The above parameter values chosen in the the code provided in the forthcoming section are less in order to result in a very short simulation run. However, for meaningful results, try increasing the number of packet errors and the maximum number of packets.

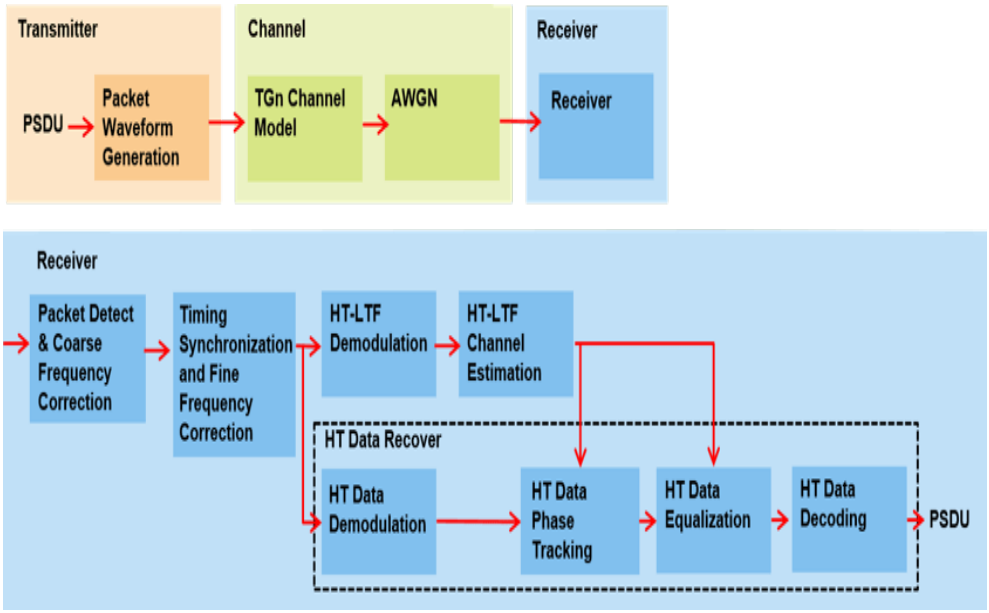


Figure 1: 802.11n Packet Error Rate simulation for 2x2 TGn Channel

MCS Chart

MCS Index	Modul-ation	Spatial Streams	802.11n Data Rate			
			20 MHz		40 MHz	
			L-GI	S-GI	L-GI	S-GI
0	BPSK	1	6.5	7.2	13.5	15
1	QPSK	1	13	14.4	27	30
2	QPSK	1	19.5	21.7	40.5	45
3	16-QAM	1	26	28.9	54	60
4	16-QAM	1	39	43.3	81	90
5	64-QAM	1	52	57.8	108	120
6	64-QAM	1	58.5	65	122	135
7	64-QAM	1	**65**	**72.2**	**135**	**150**
8	BPSK	2	13	14.4	27	30
9	QPSK	2	26	28.9	54	60
10	QPSK	2	39	43.3	81	90
11	16-QAM	2	52	57.8	108	120
12	16-QAM	2	78	86.7	162	180
13	64-QAM	2	104	116	216	240
14	64-QAM	2	117	130	243	270
15	64-QAM	2	**130**	**144**	**270**	**300**

Maximum with 1 spatial stream

Maximum with 2 spatial streams

Figure 2: Modulation and Coding Scheme chart

For each SNR point a number of packets are tested and the packet error rate calculated.

For each packet the following processing steps occur:

a. A PSDU is created and encoded to create a single packet waveform.

b. The waveform is passed through a different realization of the TGn channel model.

c. AWGN is added to the received waveform to create the desired average SNR per active subcarrier after OFDM demodulation.

d. The packet is detected.

e. Coarse carrier frequency offset is estimated and corrected.

f. Fine timing synchronization is established. The L-STF, L-LTF and L-SIG samples are provided for fine timing to allow for packet detection at the start or end of the L-STF.

g. Fine carrier frequency offset is estimated and corrected.

h. The HT-LTF is extracted from the synchronized received waveform. The HT-LTF is OFDM demodulated and channel estimation is performed.

   HT Data field is extracted from the synchronized received waveform.

i. The PSDU is recovered using the extracted field and the channel estimate.

4 Matlab Code

In this section, we present the MATLAB codes that in turn can be used to simulate the 802.11n packet error rate. For modulation and coding scheme (MCS), refer to Fig. 2.
Refer to [1], for additional information about 802.11n.

```
clc;
clear all;
% Create a format configuration object for a 2-by-2 HT
transmission
cfgHT = wlanHTConfig;
cfgHT.ChannelBandwidth = 'CBW20'; % 20 MHz channel
bandwidth
cfgHT.NumTransmitAntennas = 2;    % 2 transmit antennas
cfgHT.NumSpaceTimeStreams = 2;    % 2 space-time streams
cfgHT.PSDULength = 1000;          % PSDU length in bytes
cfgHT.MCS = 15;                   % 2 spatial streams, 64-
QAM rate-5/6
cfgHT.ChannelCoding = 'BCC';      % BCC channel coding

% Create and configure the channel
tgnChannel = wlanTGnChannel;
tgnChannel.DelayProfile = 'Model-B';
tgnChannel.NumTransmitAntennas = cfgHT.NumTransmitAntennas;
tgnChannel.NumReceiveAntennas = 2;
tgnChannel.TransmitReceiveDistance = 10; % Distance in
meters for NLOS
tgnChannel.LargeScaleFadingEffect = 'None';

snr = 10:5:50;

maxNumPEs = 200; % The maximum number of packet errors at
an SNR point
maxNumPackets = 10000; % Maximum number of packets at an
SNR point

% Get the baseband sampling rate
fs = wlanSampleRate(cfgHT);

% Get the OFDM info
ofdmInfo = wlanHTOFDMInfo('HT-Data',cfgHT);

% Set the sampling rate of the channel
tgnChannel.SampleRate = fs;

% Indices for accessing each field within the time-domain
packet
ind = wlanFieldIndices(cfgHT);
```

```
S = numel(snr);
packetErrorRate = zeros(S,1);
%parfor i = 1:S % Use 'parfor' to speed up the simulation
for i = 1:S % Use 'for' to debug the simulation
    % Set random substream index per iteration to ensure
that each
    % iteration uses a repeatable set of random numbers
    stream = RandStream('combRecursive','Seed',0);
    stream.Substream = i;
    RandStream.setGlobalStream(stream);

    % Create an instance of the AWGN channel per SNR point
simulated
    awgnChannel = comm.AWGNChannel;
    awgnChannel.NoiseMethod = 'Signal to noise ratio
(SNR)';
    % Normalization
    awgnChannel.SignalPower =
1/tgnChannel.NumReceiveAntennas;
    % Account for energy in nulls
    awgnChannel.SNR = snr(i)-
10*log10(ofdmInfo.FFTLength/ofdmInfo.NumTones);

    % Loop to simulate multiple packets
    numPacketErrors = 0;
    n = 1; % Index of packet transmitted
    while numPacketErrors<=maxNumPEs && n<=maxNumPackets
        % Generate a packet waveform
        txPSDU = randi([0 1],cfgHT.PSDULength*8,1); %
PSDULength in bytes
        tx = wlanWaveformGenerator(txPSDU,cfgHT);

        % Add trailing zeros to allow for channel filter
delay
        tx = [tx; zeros(15,cfgHT.NumTransmitAntennas)];
%#ok<AGROW>

        % Pass the waveform through the TGn channel model
        reset(tgnChannel); % Reset channel for different
realization
        rx = tgnChannel(tx);

        % Add noise
        rx = awgnChannel(rx);
```

```
        % Packet detect and determine coarse packet offset
        coarsePktOffset =
wlanPacketDetect(rx,cfgHT.ChannelBandwidth);
        if isempty(coarsePktOffset) % If empty no L-STF
detected; packet error
            numPacketErrors = numPacketErrors+1;
            n = n+1;
            continue; % Go to next loop iteration
        end

        % Extract L-STF and perform coarse frequency offset
correction
        lstf =
rx(coarsePktOffset+(ind.LSTF(1):ind.LSTF(2)),:);
        coarseFreqOff =
wlanCoarseCFOEstimate(lstf,cfgHT.ChannelBandwidth);
        rx = helperFrequencyOffset(rx,fs, -coarseFreqOff);

        % Extract the non-HT fields and determine fine
packet offset
        nonhtfields =
rx(coarsePktOffset+(ind.LSTF(1):ind.LSIG(2)),:);
        finePktOffset =
wlanSymbolTimingEstimate(nonhtfields,...
            cfgHT.ChannelBandwidth);

        % Determine final packet offset
        pktOffset = coarsePktOffset+finePktOffset;

        % If packet detected outwith the range of expected
delays from the
        % channel modeling; packet error
        if pktOffset>15
            numPacketErrors = numPacketErrors+1;
            n = n+1;
            continue; % Go to next loop iteration
        end

        % Extract L-LTF and perform fine frequency offset
correction
        lltf = rx(pktOffset+(ind.LLTF(1):ind.LLTF(2)),:);
        fineFreqOff =
wlanFineCFOEstimate(lltf,cfgHT.ChannelBandwidth);
```

```
        rx = helperFrequencyOffset(rx,fs,-fineFreqOff);

        % Extract HT-LTF samples from the waveform,
demodulate and perform
        % channel estimation
        htltf =
rx(pktOffset+(ind.HTLTF(1):ind.HTLTF(2)),:);
        htltfDemod = wlanHTLTFDemodulate(htltf,cfgHT);
        chanEst =
wlanHTLTFChannelEstimate(htltfDemod,cfgHT);

        % Extract HT Data samples from the waveform
        htdata =
rx(pktOffset+(ind.HTData(1):ind.HTData(2)),:);

        % Estimate the noise power in HT data field
        nVarHT = htNoiseEstimate(htdata,chanEst,cfgHT);

        % Recover the transmitted PSDU in HT Data
        rxPSDU =
wlanHTDataRecover(htdata,chanEst,nVarHT,cfgHT);

        % Determine if any bits are in error, i.e. a packet
error
        packetError = any(biterr(txPSDU,rxPSDU));
        numPacketErrors = numPacketErrors+packetError;
        n = n+1;
    end

    % Calculate packet error rate (PER) at SNR point
    packetErrorRate(i) = numPacketErrors/(n-1);
    disp(['SNR ' num2str(snr(i))...
          ' completed after '  num2str(n-1) ' packets,'...
          ' PER: ' num2str(packetErrorRate(i))]);
end

figure;
semilogy(snr,packetErrorRate,'-ob');
grid on;
xlabel('SNR [dB]');
ylabel('PER');
title('802.11n 20MHz, MCS15, Direct Mapping, 2x2 Channel
Model B-NLOS');
```

5 Results and Discussion

```
SNR 10 completed after 201 packets, PER: 1
SNR 15 completed after 201 packets, PER: 1
SNR 20 completed after 201 packets, PER: 1
SNR 25 completed after 218 packets, PER: 0.92202
```

Figure 3:

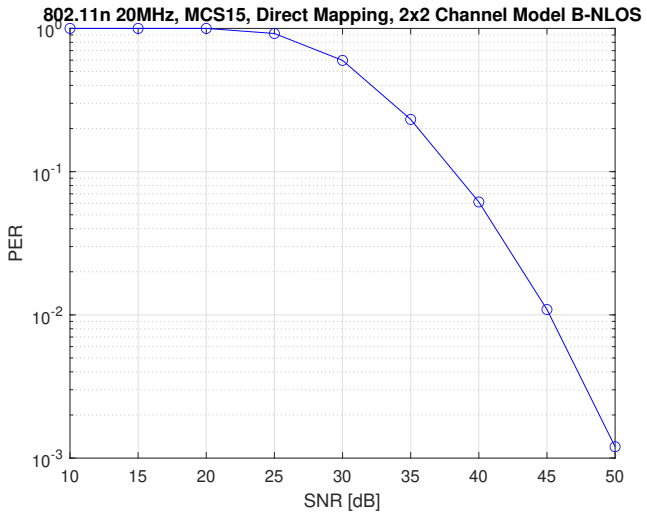


Figure 4:

References

[1] "Ieee std 802.11-2012 ieee standard for information technology - telecommunications and information exchange between systems - local and metropolitan area networks - specific requirements - part 11: Wireless LAN Medium Access Control (mac) and Physical Layer (PHY) specifications."

Simulation of MIMO multipath fading channels based on the IEEE 802.11n channel Models for indoor wireless local area networks (WLAN).

Date Performed:
Exp. No.:
Reg. No.:
Name:

1 Objective

To study MIMO multipath fading channels based on the IEEE 802.11n channel Models for indoor wireless local area networks (WLAN) with the aid of simulations based on MATLAB or IT++ library.

2 Equipments/Tool(s) Required

MATLAB OR IT++

3 Theory

The wlanTGnChannel System object™ filters an input signal through an 802.11n™ (TGn) multipath fading channel. The fading processing assumes the same parameters for all NT-by-NR links of the

TGn channel. NT is the number of transmit antennas and NR is the number of receive antennas. Each link comprises all multipaths for that link. To filter an input signal using a TGn multipath

fading channel:

- Create the wlanTGnChannel object and set its properties.
- Call the object with arguments as if it were a function.

tgn = wlanTGnChannel creates a TGn fading channel System object, tgn. This object filters a real or complex input signal through the TGn channel to obtain the channel-impaired signal.

tgn = wlanTGnChannel(Name, Value) creates a TGn channel object, tgn, and sets properties using one or more name-value pairs. Enclose each property name in quotes.

For example, wlanTGnChannel('NumReceiveAntennas',2,'SampleRate',10e6) creates a TGn channel with two receive antennas and a 10 MHz sample rate. y = tgn(x) filters input signal x through the

TGn fading channel defined by the wlanTGnChannel System object, tgn, and returns the result in y.

[y,pathGains] = tgn(x) also returns in pathGains the TGn channel path gains of the underlying fading process.

This syntax applies when you set the PathGainsOutputPort property to 1 (true). pathGains = tgac(x) returns the path gains. The NumSamples property determines the duration of the fading process. This syntax applies when you set the ChannelFiltering property to 0 (false).
The 802.11n channel object uses a filtered Gaussian noise model in which the path delays, powers, angular spread, angles of arrival, and angles of departure are determined empirically. The specific modeling approach is described in [1].

3.1 Multipath Parameters

The 802.11n channel object uses a filtered Gaussian noise model in which the path delays, powers, angular spread, angles of arrival, and angles of departure are determined empirically. The specific modeling approach is described in [**?**]. The channel is modeled as several clusters, each of which represents an independent propagation path between the transmitter and the receiver. A cluster is composed of subpaths, or taps, which share angular spreads, angles of arrival, and angles of departure. Delay and power level vary from tap to tap. Within the TGn model, clusters comprise 1–7 taps. The cluster parameters for cluster 1 of model B are shown in the table.

Parameter	Tap				
	1	2	3	4	5
Delay (ns)	0	10	20	30	40
Power (dB)	0	−5.4	−10.8	−16.2	−21.7
Angle of arrival (°)	4.3	4.3	4.3	4.3	4.3
Receiver angular spread (°)	14.4	14.4	14.4	14.4	14.4
Angle of departure (°)	225.1	225.1	225.1	225.1	225.1
Transmitter angular spread (°)	14.4	14.4	14.4	14.4	14.4

For each model, the first tap has a line of sight (LOS) between the transmitter and receiver, whereas all other taps are non-line of sight (NLOS). As a result, the first tap exhibits Rician behavior, while the others exhibit Rayleigh behavior. The Rician K-factor is the ratio between the power in the first tap and the power in the other taps. A large K-factor indicates a strong LOS component.
The angles of arrival and departure for each cluster are randomly selected from a uniform distribution over $[0, 2\pi]$. These angles are independent of each other and are fixed for all channel realizations. By fixing the values, the transmit and receive correlation matrices are computed only once. Angular spread values were indirectly determined from empirical data and fall within the 20° to 40° range. In indoor environments, the transmitter and receiver are stationary, and Doppler effects arise from people moving between them. The TGn model employs a bell-shaped Doppler spectrum in which the environmental speed, v_0, is 1.2 km/h by default (it is specified by the EnvironmentalSpeed property). The Doppler spread, fd, is calculated as$f_d = v_0/\lambda$, where λ is the carrier wavelength.

3.2 Path Loss and Shadowing

The path loss exponent and the standard deviation of the shadow fading loss characterize each model. The two parameters depend on the presence of a LOS between the transmitter and receiver. For paths with a transmitter-to-receiver distance, d, less than the breakpoint distance, dBP, the LOS parameters apply. For d ¿dBP, the NLOS parameters apply. The table summarizes the path loss and shadow fading parameters.

Parameter	Model					
	A	B	C	D	E	F
Breakpoint distance, d_{BP} (m)	5	5	5	10	20	30
Path loss exponent for $d \leq d_{BP}$	2	2	2	2	2	2
Path loss exponent for $d > d_{BP}$	3.5	3.5	3.5	3.5	3.5	3.5
Shadow fading σ (dB) for $d \leq d_{BP}$	3	3	3	3	3	3
Shadow fading σ (dB) for $d > d_{BP}$	4	4	5	5	6	6

3.3 Doppler Effects

In indoor environments, the transmitter and receiver are stationary, and Doppler effects arise from people moving between them. The TGn model employs a bell-shaped Doppler spectrum in which the environmental speed, v_0, is 1.2 km/h by default (it is specified by the EnvironmentalSpeed property). The Doppler spread, f_d, is calculated as $f_d = v_0/\lambda$, where λ is the carrier wavelength. The channel sampling rate, F_s, must be lower than the input sampling rate to avoid aliasing. It is calculated as:

$$F_s = (v_0 \times F_c)/(300 \times c)$$

where F_c is the carrier frequency, specified by the CarrierFrequency property, c is the speed of light and v_0 is defined in m/s.
In addition to basic Doppler effects resulting from environmental motion, fluorescent lights introduce signal fading at twice the power line frequency. The effects show up as frequency-selective amplitude modulation. Again, to avoid aliasing, the Nyquist frequency of the first interpolation factor must be greater than the highest harmonic.
The effect is included in models D and E. To disable this effect, set the FluorescentEffect property to false.

4 Matlab code

%Create an HT waveform having four transmit antennas and two space-time streams.

```
cfg = wlanHTConfig('NumTransmitAntennas',4,'NumSpaceTimeStreams',2, ...
    'SpatialMapping','Fourier');
txSig = wlanWaveformGenerator([1;0;0;1],cfg);
```

%Create a 4x2 MIMO TGn channel and disable large-scale fading effects.

```
tgnChan = wlanTGnChannel('SampleRate',20e6, ...
    'NumTransmitAntennas',4, ...
    'NumReceiveAntennas',2, ...
    'LargeScaleFadingEffect','None');
```

%Pass the transmit waveform through the channel.

```
rxSig = tgnChan(txSig);
```

%Display the spectrum of the two received space-time streams.

```
saScope = spectrumAnalyzer(SampleRate=20e6, ...
    ShowLegend=true, ...
    ChannelNames={'Stream 1','Stream 2'});
saScope(rxSig)
```

%Recover HT Data from 2x2 MIMO Channel

%Transmit an HT-LTF and an HT data field through a noisy 2x2 MIMO channel. Demodulate the received %HT-LTF to estimate the channel coefficients. Recover the HT data and determine the number of bit %errors.

%Set the channel bandwidth and corresponding sample rate.

```
bw = 'CBW40';
fs = 40e6;
```

%Create HT-LTF and HT data fields having two transmit antennas and two space-time streams.

```
cfg = wlanHTConfig('ChannelBandwidth',bw, ...
    'NumTransmitAntennas',2,'NumSpaceTimeStreams',2);
txPSDU = randi([0 1],8*cfg.PSDULength,1);
txLTF = wlanHTLTF(cfg);
txDataSig = wlanHTData(txPSDU,cfg);
```

%Create a 2x2 MIMO TGn channel with path loss and shadowing enabled.

```
tgnChan = wlanTGnChannel('SampleRate',fs, ...
    'NumTransmitAntennas',2,'NumReceiveAntennas',2, ...
    'LargeScaleFadingEffect','None');
```

%Create AWGN channel noise, setting SNR = 15 dB.

```
chNoise = comm.AWGNChannel('NoiseMethod','Signal to noise ratio (SNR)',...
    'SNR',15);
```

%Pass the signals through the TGn channel and noise models.

```
rxLTF = chNoise(tgnChan(txLTF));
rxDataSig = chNoise(tgnChan(txDataSig));
```

%Create an AWGN channel for a 40 MHz channel with a 9 dB noise figure. The noise variance, `nVar`, is %equal to *kTBF*, where *k* is Boltzmann's constant, *T* is the ambient temperature of 290 K, *B* is the %bandwidth (sample rate), and *F* is the receiver noise figure.

```
nVar = 10^((-228.6 + 10*log10(290) + 10*log10(fs) + 9)/10);
awgnChan = comm.AWGNChannel('NoiseMethod','Variance','Variance',nVar);
```

%Pass the signals through the channel.

```
rxLTF = awgnChan(rxLTF);
rxDataSig = awgnChan(rxDataSig);
```

%Demodulate the HT-LTF. Use the demodulated signal to estimate the channel coefficients.

```
dLTF = wlanHTLTFDemodulate(rxLTF,cfg);
chEst = wlanHTLTFChannelEstimate(dLTF,cfg);
```

%Recover the data and determine the number of bit errors.

```
rxPSDU = wlanHTDataRecover(rxDataSig,chEst,nVar,cfg);
numErr = biterr(txPSDU,rxPSDU)
```

%Compare Random Number Sources for TGn Channel

%Create a non-HT configuration object with default parameters. Generate a waveform for the configuration.

```
cfg = wlanNonHTConfig;
tx = wlanWaveformGenerator([1;0;0;1],cfg);
```

%Create a TGn channel System object with default parameters. Display the value of
%the `RandomStream` property.

```
tgnChan = wlanTGnChannel;
disp(tgnChan.RandomStream)
```

```
Global stream
```

%Pass the waveform through the channel twice, resetting the System object between the two iterations.

```
for i = 1:2
    rx(:,i) = tgnChan(tx);
    reset(tgnChan);
end
```

%Compare the two received waveforms. They are different because the `reset` object function resets the %filters and the channel object takes new random numbers from the global stream. This causes it to %generate a different channel realization.

```
isequal(rx(:,1),rx(:,2))
```

%Now release the System object and set the `RandomStream` property to `"mt19937ar with seed"`.

```
%release(tgnChan);
tgnChan.RandomStream = "mt19937ar with seed";
```

%Pass the waveform through the channel twice, resetting the System object between the two iterations.

```
for i = 1:2
    rx(:,i) = tgnChan(tx);
    reset(tgnChan);
end
```

5 Results and Discussion

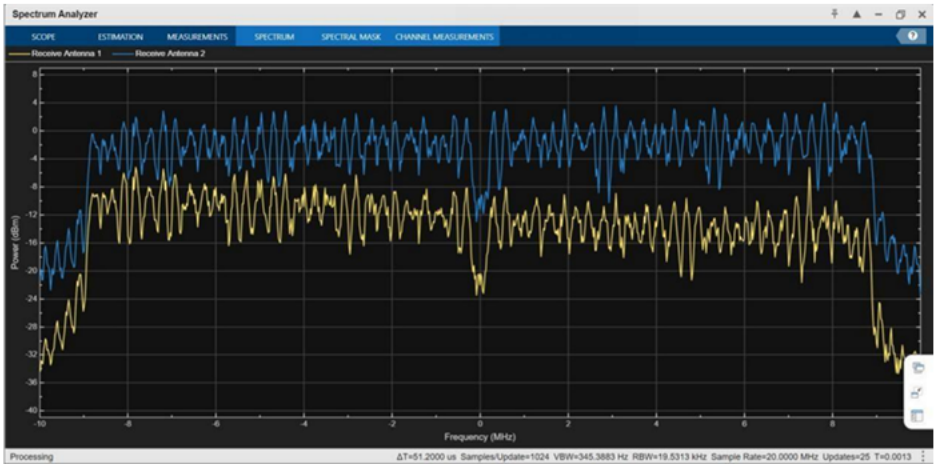


Figure 1: Simulation of MIMO multipath fading channels based on the IEEE 802.11n channel Models for indoor wireless local area networks (WLAN).

References

[1] L. S. P. K. e. a. Erceg, V., "Tgn channel models. version 4. ieee 802.11-03/940r4,," May 2004.

Simulation of MIMO multipath fading channels based on the IEEE 802.16 channel Models for fixed wireless applications

Date Performed:
Exp. No.:
Reg. No.:
Name:

1 Objective

To study MIMO multipath fading channels based on the IEEE 802.16 channel Models for fixed wireless applications with the aid of simulations based on MATLAB or IT++ library.

2 Equipments/Tool(s) Required

MATLAB OR IT++

3 Theory

3.1 WiMAX Channel Model

The WiMAX (IEEE® 802.16) channel models [1] for fixed wireless applications are proposed for scenarios where the cell radius is less than 10 km, the directional antennas at the receiver are installed under-the-eaves/windows or on the rooftop, and the base station (BS) antennas are 15 to 40 m in height. The channel models comprise a set of path loss models including shadowing (suburban, urban) and a multipath fading model, which describes the multipath delay profile, the K-factor distribution, and the Doppler spectrum. The antenna gain reduction factor, due to the use of directional antennas, is also characterized.
In this study, we use a MIMO multipath fading channel System object™
comm.MIMOChannel with two transmit antennas, one receive antenna and a rounded Doppler spectrum structure. The modified Stanford University Interim (SUI) channel models consist of a set of 6 typical channels used to simulate the IEEE 802.16 channel models (more specifically the 2004 version of the standard for fixed wireless applications). They are proposed for a scenario where: the cell size is 7 km, the BS antenna height is 30 m, the receive antenna height is 6 m, the BS antenna beamwidth is 120 degrees, the receive antenna is either omnidirectional or directional (30 degrees), and only vertical polarization is used.
Each modified SUI channel model has three taps. Each tap is characterized by a relative delay (concerning the first path delay), a relative power, a Rician K-factor, and a maximum Doppler shift. Two sets of relative powers are specified for each channel model: one for an omnidirectional antenna, and one for a 30-degree directional antenna. Furthermore, for each set of relative powers, two different K-factors are specified, a K-factor for 90% cell coverage, and a K-factor for 75% cell coverage. Hence, each of the 6 modified SUI channel models comprises parameters for four distinct scenarios.

Each modified SUI channel model is further assigned an antenna correlation, defined as the envelope correlation coefficient between signals received at different antenna elements.
The code below constructs a MIMO fading channel System object according to the modified SUI-1 channel model, for an omnidirectional antenna and 90% cell coverage.
The channel model has 3 paths: the first path is Rician while the remaining two are Rayleigh. Each path has a rounded Doppler spectrum for its diffuse component: the parameters are as specified in the doppler('Rounded') structure. While different maximum Doppler shifts are specified for each path in [1], we use the maximum value of the Doppler shifts for all paths.
We use 2 transmit antennas and 1 receive antenna. Similar to Appendix B of [1], the correlation coefficient between the two links on each path is taken equal to the antenna spatial correlation. The correlation coefficient is 0.7.
The sample rate for a WiMAX system is 1.429, 2.857, 5.714, 11.429 or 22.857 MHz. At such rates with a small Doppler shift, we need many samples and a long simulation time to sufficiently exhibit the channel's statistical characteristics. To avoid that, we arbitrarily choose a smaller sample rate of 0.1 MHz. You can increase the sample rate, Rsamp, and number of samples, Ns, at the same time to see similar statistical results.

4 Matlab code

```
Rsamp = 0.1e6;
Ns = 3e6;

wimaxChan = comm.MIMOChannel( ...
    'SampleRate', Rsamp, ...
    'PathDelays', [0 0.4 0.9]*1e-6, ...
    'AveragePathGains', [0 -15 -20], ...
    'FadingDistribution', 'Rician', ...
    'KFactor', 4, ...
    'MaximumDopplerShift', .5, ...
    'DopplerSpectrum', doppler('Rounded'), ...
    'TransmitCorrelationMatrix', [1 0.7; 0.7 1], ...
    'ReceiveCorrelationMatrix', 1, ...
    'PathGainsOutputPort', true);
```

%Simulate the modified SUI-1 channel model with a long QPSK modulated frame input

```
Nt = size(wimaxChan.TransmitCorrelationMatrix, 1);
x = pskmod(randi([0 3], Ns, Nt), 4);
[~, g] = wimaxChan(x);
```

%Doppler spectrum of the 1st link of the second path is estimated from the complex path gains and plotted.

```
figure;
win = hamming(Ns/5);
Noverlap = Ns/10;
pwelch(g(:,2,1),win,Noverlap,[],Rsamp,'centered')
axis([-0.1/10 0.1/10 -80 10]);
legend('Simulation');
```

%The theoretical rounded Doppler spectrum is overlaid on the estimated Doppler spectrum.

%Observe a good fit between them.

```
fd = wimaxChan.MaximumDopplerShift;
f  = -fd:0.01:fd;
a  = wimaxChan.DopplerSpectrum.Polynomial;      % Parameters of the rounded Doppler
spectrum
Sd = 1/(2*fd*(a(1)+a(2)/3+a(3)/5))*(a(1)+a(2)*(f/fd).^2+a(3)*(f/fd).^4);
Sd = Sd*10^(wimaxChan.AveragePathGains(2)/10);  % Scaling by average path power

hold on;
plot(f(Sd>0)/1e3,10*log10(Sd(Sd>0)),'k--');
legend('Simulation','Theory');
```

%The Doppler spectrum for the 2nd link of the 2nd path is also estimated and compared to the theoretical spectrum

%Observe a good fit between them.

```
figure;
```

```
pwelch(g(:,2,2),win,Noverlap,[],Rsamp,'centered')
axis([-0.1/10 0.1/10 -80 10]);
legend('Simulation');
hold on;
plot(f(Sd>0)/1e3,10*log10(Sd(Sd>0)),'k--');
legend('Simulation','Theory');
```

%For each path, we plot the fading envelope waveforms of both transmit links. We can observe a correlation between the fading envelopes.

```
figure;
semilogy(abs(g(:,1,1)),'b');
hold on;
grid on;
semilogy(abs(g(:,1,2)),'r');
legend('First transmit link','Second transmit link');
title('Fading envelopes for two transmit links of Path 1');

figure;
semilogy(abs(g(:,2,1)),'b');
hold on;
grid on;
semilogy(abs(g(:,2,2)),'r');
legend('First transmit link','Second transmit link');
title('Fading envelopes for two transmit links of Path 2');

figure;
semilogy(abs(g(:,3,1)),'b');
hold on;
grid on;
semilogy(abs(g(:,3,2)),'r');
legend('First transmit link','Second transmit link');
title('Fading envelopes for two transmit links of Path 3');
```

%Compute the spatial correlation matrices for each path. We observe that they show a match with the %theoretical values Rt. Note that `corrcoef` function estimate can be improved if `Ns` is increased.

```
TxCorrMatrixPath1 = corrcoef(g(:,1,1),g(:,1,2)).'
TxCorrMatrixPath2 = corrcoef(g(:,2,1),g(:,2,2)).'
TxCorrMatrixPath3 = corrcoef(g(:,3,1),g(:,3,2)).'
```

5 Results and Discussion

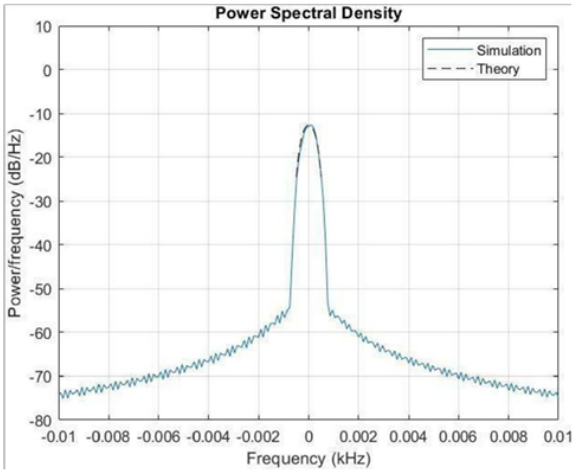


Figure 1: The Doppler spectrum for the 1st link of the 1st path

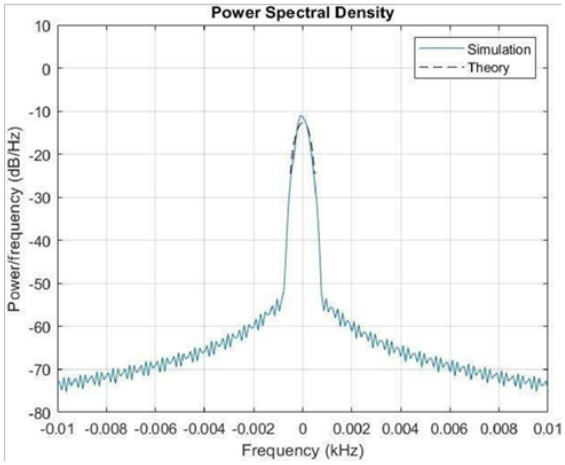


Figure 2: The Doppler spectrum for the 2nd link of the 2nd path

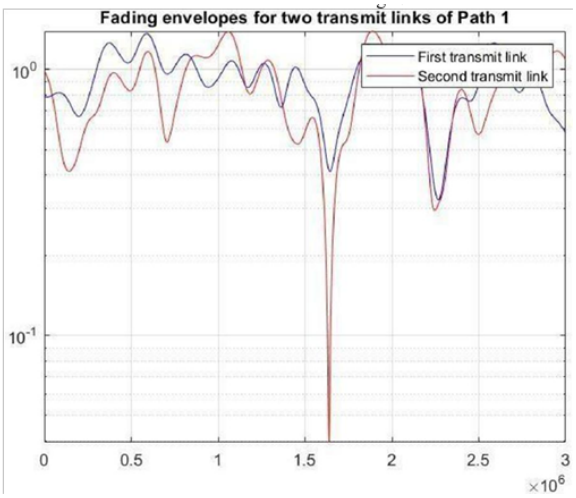


Figure 3: Fading envelopes for the two transmit links of Path 1

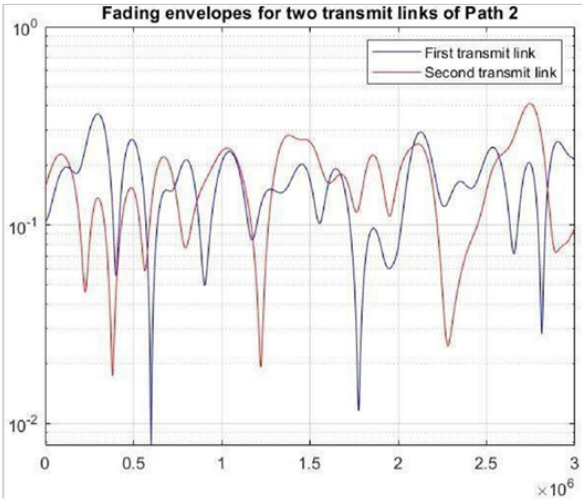


Figure 4: Fading envelopes for two transmit links of Path 2



Figure 5: Fading envelopes for two transmit links of Path 3

References

[1] "IEEE 802.16 broadband wireless access working group, "channel models for fixed wireless applications", ieee 802.16a-03/01,," vol. 2003-06-27.